A Light-Following (Phototaxis) Robot

Zhanuzak Sayassat, Begmanov Arman, Satymkul Farabi

School of IT and Engineering
Kazakh-British Technical University
Almaty, Kazakhstan

s_zhanuzak@kbtu.kz, f_satymkul@kbtu.kz, a_begmanov@kbtu.kz

Abstract

Background: Phototaxis robots are widely introduced as an example of a simple sensor usage to a car robot. The functionality of the robot is simple: it follows the brightest spot in the environment.

Problem: Designing a light-following robot is a challenge due to sensor noise, hardware imperfections, and speed regulations of wheels while turning sides.

Method: We developed a differential-drive robot using an ESP32 microcontroller, an ultrasonic sensor, and two photoresistors. A proportional-derivative (PD) control algorithm was implemented to minimize the intensity difference between sensors and move the robot toward the light source.

Results: PID control showed the best stability. Our robot operates as needed.

Conclusion: The PD controller with gain scheduling provided sufficient stability and responsiveness for real-time phototaxis. Future improvements include sensor filtering and adaptive control strategies.

Keywords: phototaxis, ESP32, PD control, embedded systems, robotics.

1 Introduction

Autonomous navigation based on environmental sensing is a fundamental problem in robotics.

The main challenge is achieving stable motion toward a light source while handling sensor noise and actuator delay.

The objective of this project is to design and implement a mobile robot capable of following a light source using feedback control.

2 Related Works

Phototaxis robots have been widely studied using simple analog circuits and digital control systems. Early times relied on mapping between sensor input and motor output, resulting in unstable behavior.

More advanced systems, especially in manufacturing processes, apply PID or PI control to improve stability. However, full PID is often not needed for simple control tasks.

In this work, we implement a PD controller that balances responsiveness and stability without introducing integral windup.

3 Methodology

3.1 Hardware Architecture

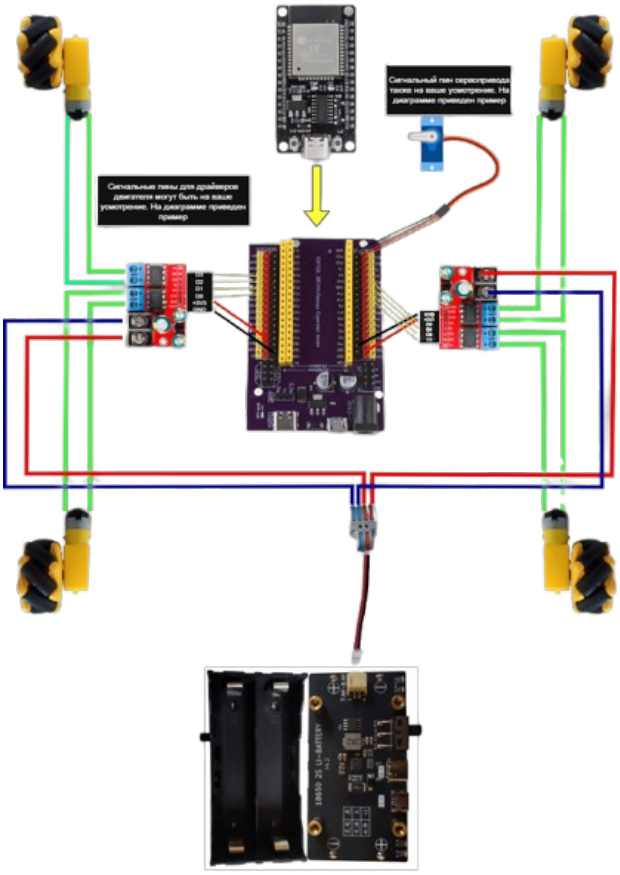


Figure 1: Assemble Scheme

Figure 1 illustrates the electrical assembly scheme of the robotic platform. The system is built around an ESP32 microcontroller connected to a motor driver shield that controls four DC motors arranged in a four-wheel-drive configuration.

Two dual-channel motor driver modules are used to independently regulate the left and right motor pairs. Power is supplied through an external battery module.

The diagram also shows the connection of an ultrasonic distance sensor used for obstacle detection. The ESP32 serves as the central control unit.

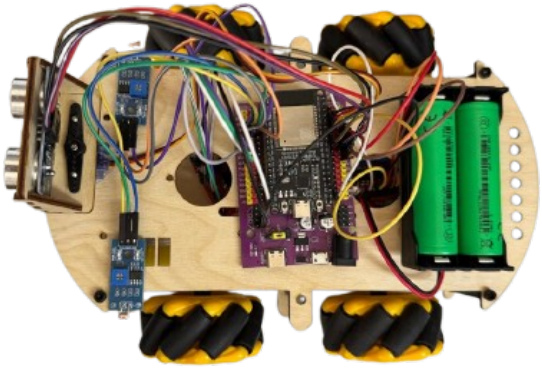


Figure 2: Robot's Hardware View from Above

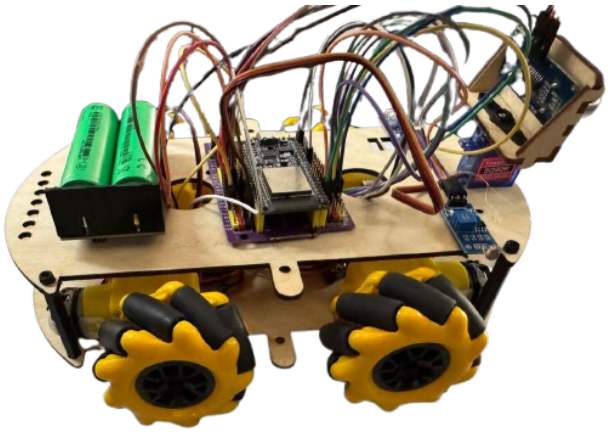
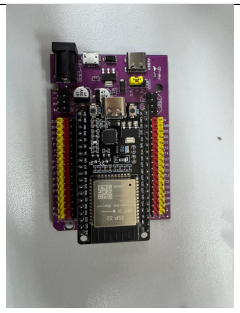
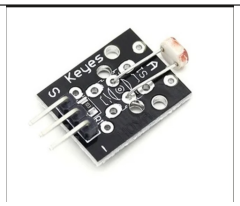
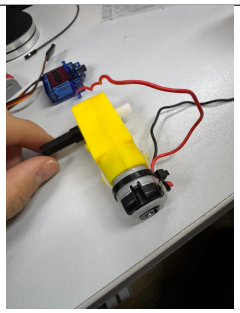
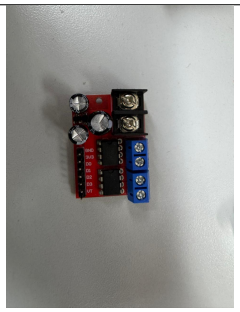


Figure 3: Robot's Hardware View from Side

The system consists of the following components:

Component	Specs	Remarks
ESP32	240 MHz, WiFi/Bluetooth	
Photoresistors (2x)	Analog light sensors with modules	
DC Motors (2x)	6V, 200 RPM	
Motor Driver (L298N)	Dual H-bridge	
Ultrasonic Sensor	5V, 2–400 cm range, 3 mm accuracy	

The two photoresistors are placed on the left and right sides of the robot. The ESP32 reads analog values from both sensors which are the raw measurements of the sensor. Then it computes the difference between two photoresistors and then uses PD control loop with gain scheduling to reduce the difference by turning the robot with according speed toward a light source.

The ultrasonic sensor ensures that the robot does not bump to a light source. This obstacle-avoiding behavior is designed for avoiding crashes of the robot.

3.2 Software Architecture

Ultrasonic sensor uses 25,000 μs waiting time for the ultrasonic echo. This time is computed using the sound speed in a room temperature (20 ° C).

$$d = \frac{343 \text{ m/s} \cdot t}{2}$$

Taking $d = 4$ m for our ultrasonic sensor characteristics:

$$t = \frac{2 \cdot 4 \text{ m}}{343 \text{ m/s}} \approx 0.0233 \text{ ms}$$

To ensure the ultrasonic wave comes to the ultrasonic sensor on time, we take the waiting time a bit longer of 0.025 ms. Those characteristics are written in the following code lines:

```
int OBS_DISTANCE = 20;
const unsigned long ECHO_TIMEOUT = 25000;
```

The loop code. Giving higher priority for the obstacle-avoiding behavior of the robot, we ensure that the robot does not crash. When an obstacle is detected, the robot drives backwards. To ensure PD control's derivative action not blow up when our robot drives backward, we immediately stop and then make some delay. This behavior is shown in the code below:

```
void loop() {
  int leftVal  = read_LDR(LDR_LEFT);
  int rightVal = read_LDR(LDR_RIGHT);

  long distance = get_distance();
  // printing information into a serial monitor
  Serial.print("L:");
  Serial.print(leftVal);
  Serial.print("R:");
  Serial.print(rightVal);
  Serial.print("D:");
  Serial.println(distance);

  // obstcle avoidance algorithm
  if (distance > 0 && distance < OBS_DISTANCE) {
```

```
    left_motor(-v_0);
    right_motor(-v_0);
    delay(300);
    motor_stop();
    \\ ensuring no derivative spike
    error_prev = 0;
    last_time = millis();
    delay(200);
    return;
  }
  PD(leftVal, rightVal); \\apply PD
  delay(20);
}
```

The function below measures the distance from the ultrasonic to an obstacle. It sends short 10-microsecond pulse. The function measures how long it takes for the echo to return and then calculates the distance using the speed of sound wave. If no echo is detected, it returns -1 to indicate no object or is far away.

```
const long NO_ECHO = -1;
long get_distance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, ECHO_TIMEOUT);

  if (duration == 0)
    return NO_ECHO;

  return duration * 0.034 / 2; // duration * speed of sound /2
}
```

As the robot uses a simple analog photoresistors, they become noisy. To smooth the sensor readings, Moving Average filter is applied:

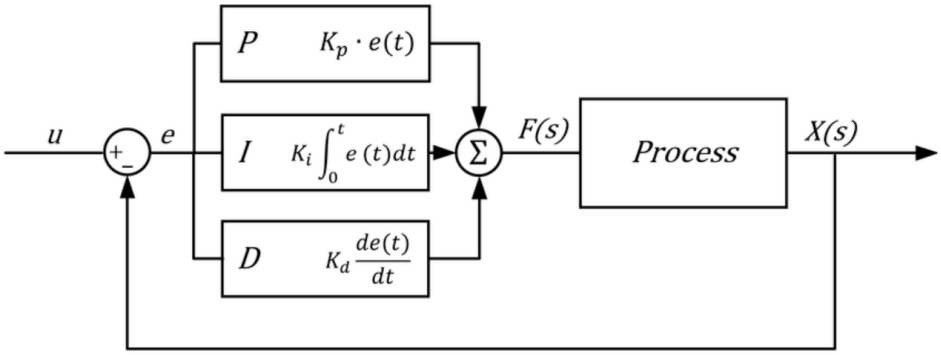


Figure 5: PID Control Scheme

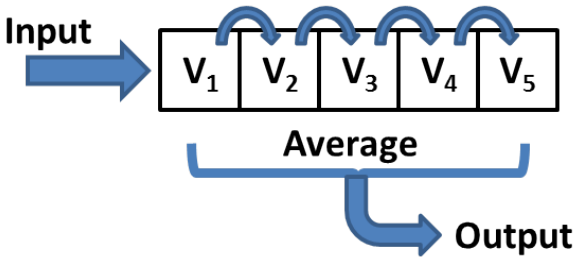


Figure 4: Moving Average

$$\text{average} = \frac{1}{N}\sum_{i=1}^{N} x_i$$

In our case, we are finding the average value of five readings:

$$\text{average} = \frac{1}{5}\sum_{i=1}^{5} x_i$$

The code below demonstrates this filtering technique:

```
int read_LDR(int pin) {
  long sum = 0;
  for (int i = 0; i < 5; i++)
  {
    sum += analogRead(pin);
  }
  return sum / 5;
}
```

The system follows a feedback loop:

The whole PID loop

$$u = K_\mathrm{p} e + K_\mathrm{d}\frac{de}{dt} + K_\mathrm{i}\int e(\tau)\,d\tau$$

The photoresistor sensors measure light from both left and right sides. We are then measuring the time since ESP32 code started running. Setting $dt \approx \Delta t = t_{\text{now}} - t_{\text{last time}}$ The Raw Error is the the difference between values of the photoresistors and is calculated by:

$$e = x_{\text{right value}} - x_{\text{left value}}$$

The PD control loop involves proportional and derivative terms:

$$u = K_{\text{p}} e + K_{\text{d}} \frac{de}{dt}$$

Expanding derivative term:

So overall PD control loop becomes:

$$u = K_{\text{p}} e + K_{\text{d}} \frac{e - e_{\text{prev}}}{t_{\text{now}} - t_{\text{last time}}}$$

From our observation, the robot does not properly turn left or right when a light source is very close to the sensors. In that case, we applied a method Gain Scheduling. In our case, the gain scheduling is defined as the following piece-wise function:

$$GS = \begin{cases} 1, & e \geq 200 \\ 1.5, & e < 200 \end{cases}$$

Overall, PD control loop is

$$u' = GS \cdot \left(K_{\text{p}} e + K_{\text{d}} \frac{e - e_{\text{prev}}}{t_{\text{now}} - t_{\text{last time}}} \right) \quad \text{or}$$

$$u' = \begin{cases} K_{\text{p}} e + K_{\text{d}} \frac{e - e_{\text{prev}}}{t_{\text{now}} - t_{\text{last time}}}, & e \geq 200 \\ 1.5 \cdot \left(K_{\text{p}} e + K_{\text{d}} \frac{e - e_{\text{prev}}}{t_{\text{now}} - t_{\text{last time}}} \right), & e < 200 \end{cases}$$

Calculating the speeds of the right and left side wheels using PD control loop:

$$v_{\text{left}} = v_0 - u', \quad v_{\text{right}} = v_0 + u'$$

where v_0 is the baseline speed of this robot which is 100. Experimentally we adjusted tuning parameters as such:

```
int v_0 = 100;
float Kp = 0.10f;
float Kd = 0.65f;

float error_prev = 0.0f;
unsigned long last_time = 0;
```

The code below demonstrates this behavior:

```
void PD(int leftVal, int rightVal) {
  unsigned long now = millis();
  float dt = (now - last_time);
  if (dt <= 0)
    dt = 1; // no division by 0!!!

  int raw_error = rightVal - leftVal;
  float error = (float)raw_error;

  // gain scheduling (GS)
  float GS = 1.0f;
  if (abs(raw_error) < 200) {
    GS = 1.5f;
  }

  float derivative = (error - error_prev) / dt;
  float u_gs = GS * (Kp * error + Kd * derivative);

  int speed_left  = v_0 - u_gs;
  int speed_right = v_0 + u_gs;

  speed_left  = constrain(speed_left,  -255, 255); //limiting speed
  speed_right = constrain(speed_right, -255, 255); // limiting speed

  left_motor(speed_left);
  right_motor(speed_right);

  error_prev = error; // updating error
  last_time = now; // updating time
}
```

The following code shows the speed of the right and left wheels:

```
void left_motor(int speed) {
  speed = constrain(speed, -255, 255);
  int pwm = abs(speed);

  if (speed > 0) {
    ledcWrite(chL1, pwm);
```

```
    ledcWrite(chL2, 0);
    ledcWrite(chL3, 0);
    ledcWrite(chL4, pwm);
  }
  else if (speed < 0) {
    ledcWrite(chL1, 0);
    ledcWrite(chL2, pwm);
    ledcWrite(chL3, pwm);
    ledcWrite(chL4, 0);
  }
  else {
    ledcWrite(chL1, 0);
    ledcWrite(chL2, 0);
    ledcWrite(chL3, 0);
    ledcWrite(chL4, 0);
  }
}

void right_motor(int speed) {
  speed = constrain(speed, -255, 255);
  int pwm = abs(speed);
  if (speed > 0) {
    ledcWrite(chR1, 0);
    ledcWrite(chR2, pwm);
    ledcWrite(chR3, 0);
    ledcWrite(chR4, pwm);
  }
  else if (speed < 0) {
    ledcWrite(chR1, pwm);
    ledcWrite(chR2, 0);
    ledcWrite(chR3, pwm);
    ledcWrite(chR4, 0);
  }
  else {
    ledcWrite(chR1, 0);
    ledcWrite(chR2, 0);
    ledcWrite(chR3, 0);
    ledcWrite(chR4, 0);
  }
}

void motor_stop() {
  ledcWrite(chL1, 0);
  ledcWrite(chL2, 0);
  ledcWrite(chL3, 0);
  ledcWrite(chL4, 0);

```

```
54    ledcWrite(chR1, 0);
55    ledcWrite(chR2, 0);
56    ledcWrite(chR3, 0);
57    ledcWrite(chR4, 0);
58  }
```

4 Experimental Results

4.1 Experimental Setup

Experiments were conducted inside laboratory with controlled lighting. A light source was placed at varying distances. Each trial we changed proportional gain K_{p} and derivative gain K_{d} to reach best robot turning behavior. The tuning process was totally experimental:
- For a simple robot we started with the proportional gain until the system output begins oscillate.
- Then reduced K_{p} to about half of the value that caused oscillation to achieve a stable starting point.
- Tuned derivative gain K_{d} to damped overshoot and improve stability.

The proportional gain already reduces error. If the robot drifts, correction appear immediately. When the is approaching the line too quickly, the derivative becomes large and controller reduces aggresiveness. The line-following robot responses fast and cares more about stability. For this simp We do not need integral gain. It did not contribute a lot for robot stabilization.

Using Gain Schedule technique helped the robot for quick turning when the error is big but the turning speed is too low for a robot to turn precisely to the light source.

With some modifications of our code, we could extract useful information and below plotted graphs with matplotlib.pyplot.

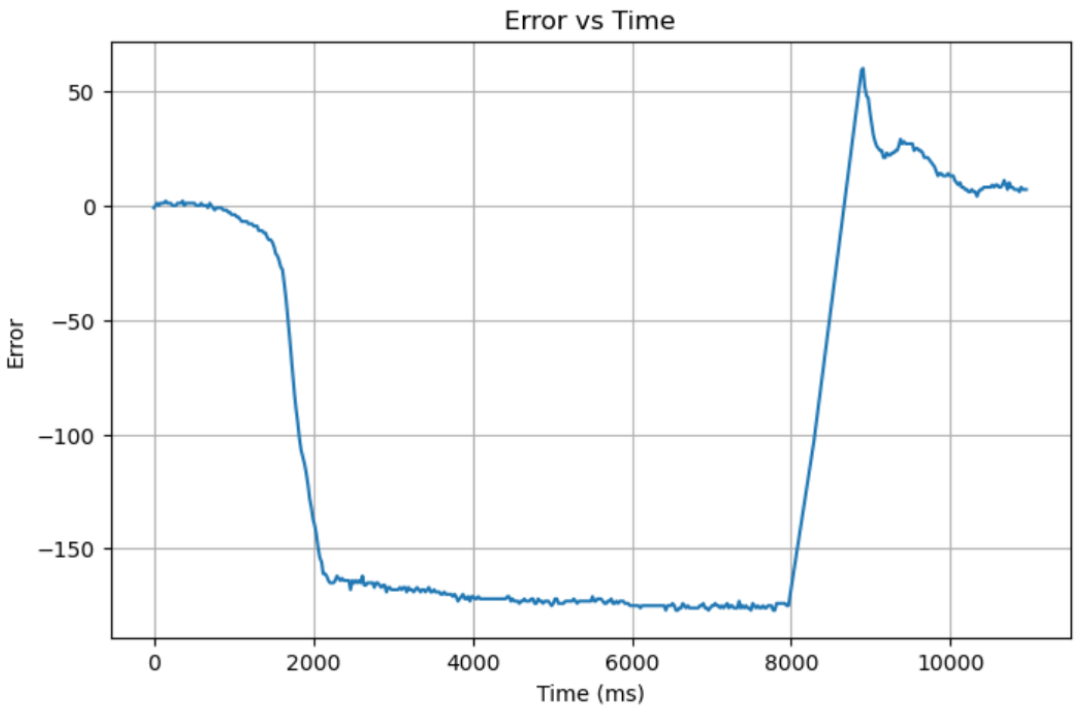


Figure 6: PD control: Error VS Time

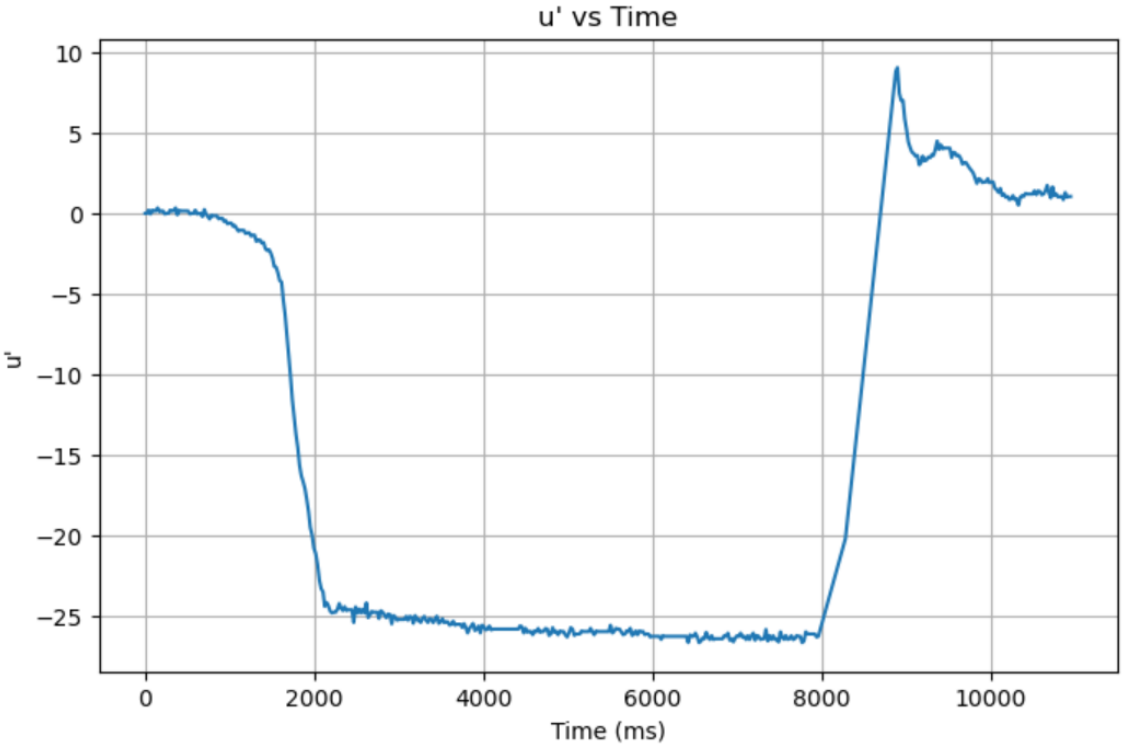


Figure 7: PD control: u' vs Time

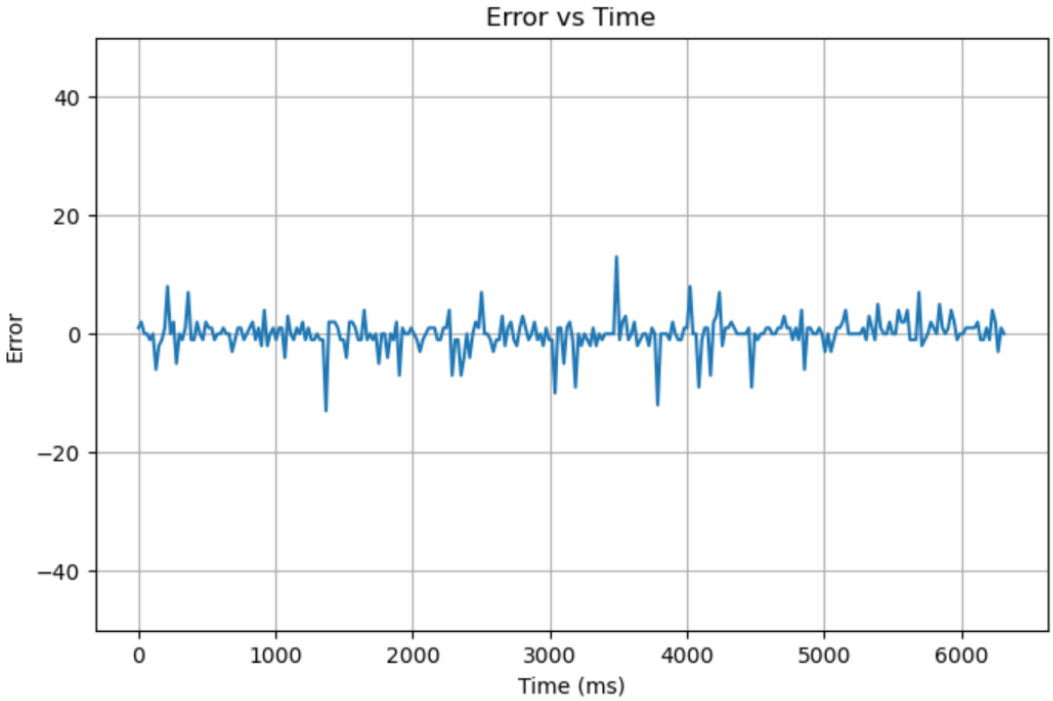


Figure 8: Full PID control

4.2 Results and Discussion

Overall, PID control performed much better than PD control which contradicts our initial assumption. The main reason for this is the integral component removes offset, reduces state-state error and gives stronger equilibrium regulation.

5 Conclusion

This project demonstrated the design and implementation of a light-following robot using an ESP32 and PD control. The system achieved high accuracy under stable conditions and showed fast response times. The PID control showed the best performance.

Future work includes implementing PID control along with mathematical calculations.

Acknowledgment

We thank the faculty of KBTU for providing laboratory resources and guidance.

References

1. Karl Johan Astrom, Richard M. Murray, Feedback Systems An Introduction for Scientists and Engineers, Second Edition

2. MUHAMMAD ILYAS, Lecture Notes